Decision Trees and Pegasos SVM for Bina.az Property Price Prediction

Esgerov Elmir Khalilli Yagub Sadigov Ruslan

MLE-AI-201 Machine Learning, AI Academy, Cohort II

***Abstract*— This project studies property asking prices in the Bina.az Sale Project dataset. It addresses two tasks: predicting price in AZN and classifying listings as premium or standard using the training-set median. Cleaning reduces 100,775 rows to 64,454 by removing repeated listing snapshots. Target-derived columns are excluded, and preprocessing statistics come from training rows only. The core models are NumPy implementations of a decision tree and a mini-batch Pegasos support-vector machine. Random Fourier features provide a non-linear SVM option. Validation selects the settings, and three-fold cross-validation measures variation within the training partition. On 9,669 test listings, the NumPy classification tree achieves F1 of 0.9068 and ROC-AUC of 0.9594, close to the scikit-learn tree. The linear and non-linear SVMs achieve F1 of 0.8553 and 0.8478. The regression tree achieves log-RMSE of 0.4102 and improves test MAE by 35.5% over a training-median predictor. However, its price-scale R^2 is only 0.0105. Extremely large source prices dominate squared error and remain in the main evaluation. These results support the implementations while showing the limits of this dataset and experimental protocol. Repository: `github.com/yagubbx/bina-az-ML-project`.**

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, real estate, regression.

I. Introduction

Property listings combine numeric characteristics, location and categorical information. They also contain repeated records, missing values and unusual prices. This makes them useful for studying both model implementation and the practical effects of data preparation. The target in this project is an *asking price*; the dataset does not establish the final transaction price.

Task A predicts price with a decision tree trained on natural-log price. Task B predicts a price tier with a classification tree and an SVM. The main contributions are a shared data pipeline, NumPy implementations of both algorithms, a non-linear SVM extension, and a comparison with matched scikit-learn baselines. The purpose is to understand model behaviour and explain errors, rather than report only the strongest metric.

II. Background

Decision trees split a feature space into regions. A classification tree selects splits that reduce class impurity, while a regression tree reduces within-node target variance. Their stopping rules limit complexity. The scikit-learn tree documentation provides the reference behaviour used in the comparison [2].

A soft-margin SVM balances a weight penalty with hinge loss. Pegasos optimizes this objective using sub-gradient updates and a decreasing learning rate [3]. Random Fourier features (RFF) approximate a shift-invariant kernel through an explicit random mapping [4]. This project uses that mapping with the same linear solver, avoiding a full training kernel matrix.

III. Data and Preprocessing

A. Dataset and cleaning decisions

The source is the Bina.az Sale Project dataset published on Kaggle [1]. Table 1 summarizes the data used here. The pipeline retains the first observed record for each listing URL in file order. This removes 36,321 repeated snapshots, leaving 64,454 rows. There are no additional exact-row duplicate removals and no rows removed by the positive-price or AZN-currency checks in this run. Different URLs may still describe the same property.

Table 1. Dataset and split summary. Premium means price strictly above 210,000 AZN.

Partition	Rows	Premium (%)
Raw file	100,775	—
After cleaning	64,454	—
Training	45,117	49.31
Validation	9,668	49.78
Test	9,669	49.79

Numeric strings are converted to numbers. Area values expressed in sot are multiplied by 100 to obtain square metres. Floor information is separated into the current floor and total number of floors. Non-positive area and room values, inconsistent floors and invalid geographic coordinates become missing values. Geographic validation checks coordinate ranges, not the correctness of each location.

Missing numeric values use training medians. Missing categories receive a dedicated value. Numeric inputs are clipped using their training 1st and 99th percentiles. The model also receives missing-value indicators and one-hot categorical columns. Unseen categories map to all-zero indicators. These steps produce 179 input features. Clipping applies to predictors, not to the target price.

B. Features and target leakage

Numeric predictors are area, rooms, floor, total floors, land area, latitude and longitude. Categorical predictors are location, city, building type, property category, repair status, bill-of-sale status and mortgage availability. Absent flags remain missing rather than becoming a confirmed negative answer. For example, 6,094 cleaned rows have missing room counts and 54,823 have missing land area.

The columns `unit_price` and `total_price` are excluded from both tasks: one is a price-derived quantity and the other restates the target. The feature whitelist also excludes names, raw addresses, listing identifiers, scraping metadata and free text. Listing URLs serve only to remove repeated records; they are not model inputs.

C. Targets, splits and exploratory analysis

A seeded permutation reserves 70% of the cleaned data for training. Its median price, 210,000 AZN, defines the premium label. The remaining data are divided into validation and test partitions stratified by that label. Thus, validation and test are stratified; the initial training reservation is random. Ties at the median belong to the standard class. No held-out price contributes to the threshold.

Exploratory plots use training data only and cover raw and log-price distributions, area versus price, room counts, coordinates and tier balance. Prices have a long upper tail. The training 90th percentile is 520,000 AZN. Location is informative but coverage is uneven. Median imputation, category vocabularies and scaling are fitted after splitting, so held-out rows cannot influence these statistics.

IV. Method

A. Decision tree

For class proportions p_k, the supported impurities are

$$G = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k. \tag{1}$$

Regression uses $I = N^{-1}\sum_i (z_i - \bar{z})^2$, where $z_i = \log(y_i)$ and y_i is price. The gain of a split into left and right children is

$$\Delta I = I_P - \frac{N_L}{N_P} I_L - \frac{N_R}{N_P} I_R. \tag{2}$$

For each feature, the implementation sorts values and tests thresholds between distinct adjacent values. Cumulative class counts or target sums allow candidate evaluation without rebuilding each child for every threshold. Recursion chooses the largest positive gain that satisfies the leaf-size constraint.

Growth stops at maximum depth, an insufficient split size, a pure or unsplittable node, or an insufficient weighted impurity decrease. The API supports `max_depth`, `min_samples_split`, `min_samples_leaf` and `min_impurity_decrease`. Experiments vary depth and leaf size, while the other two retain defaults of 2 and 0. Classification leaves return a majority class and class-frequency probabilities. Regression leaves return a mean log price, exponentiated for AZN evaluation. No retransformation bias correction is applied.

Feature importance sums sample-weighted impurity reductions for each feature and normalizes the totals. It measures use within this fitted tree, not a causal effect on price.

B. Pegasos SVM

Training labels are mapped to $y_i \in \{-1, +1\}$. With score $f_i = \mathbf{w}^\top \mathbf{x}_i + b$, the objective is

$$J = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \max(0, 1 - y_i f_i). \tag{3}$$

The implementation shuffles rows using seed 42 and processes mini-batches of at most 256 examples. At step t, let B_t be the batch and $A_t = \{i \in B_t : y_i f_i < 1\}$. With $\eta_t = 1/(\lambda t)$, the update is

$$\widetilde{\mathbf{w}} = (1 - \eta_t \lambda)\mathbf{w} + \frac{\eta_t}{|B_t|}\sum_{i \in A_t} y_i \mathbf{x}_i, \tag{4}$$

$$b \leftarrow b + \frac{\eta_t}{|B_t|}\sum_{i \in A_t} y_i, \tag{5}$$

$$\mathbf{w} \leftarrow \min\left(1, \frac{1}{\sqrt{\lambda}\|\widetilde{\mathbf{w}}\|}\right)\widetilde{\mathbf{w}}. \tag{6}$$

Projection is applied only when the norm exceeds $1/\sqrt{\lambda}$. The bias is separate and unregularized. Gradients average over the full batch, including examples with zero hinge gradient. The full training objective is recorded after every epoch; stochastic updates need not reduce it at every epoch.

Inputs use training means and standard deviations. The class prediction follows the score sign, with zero assigned to premium. The reported geometric margin width is $2/\|\mathbf{w}\|$. Points with signed margin at most one form the reported support mask. These are margin-active points under the primal model, rather than exact non-zero dual coefficients. An alternative $0.1/\sqrt{t}$ schedule is available for sensitivity analysis.

C. Non-linear extension

RFF maps standardized $\mathbf{x}$ to

$$\phi_j(\mathbf{x}) = \sqrt{2/D}\cos(\omega_j^\top \mathbf{x} + u_j), \tag{7}$$

where $\omega_j \sim \mathcal{N}(0, 2\gamma I)$ and $u_j \sim U(0, 2\pi)$. The approximate kernel is $\exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$. The mapping uses a fixed seed, and the selected configuration has $D = 512$. Both the NumPy and scikit-learn RFF models use the same mapping. This permits a comparison of solvers with shared inputs.

V. Experimental Setup

A. Selection and baselines

The regression tree search covers depths $\{3, 6, 10, 14, 18\}$ and leaf sizes $\{5, 20\}$. Classification uses depths $\{3, 6, 10\}$, both leaf sizes, and Gini or entropy. Selection minimizes validation log-RMSE for regression and maximizes validation F1 for classification.

The linear SVM search uses $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and 20 or 40 epochs. Initial RFF candidates combine these settings with $D = 128$ and $\gamma \in \{0.01, 0.1\}$. An extended validation search uses $D = 512$, 80 epochs, $\lambda \in \{10^{-4}, 10^{-3}\}$ and $\gamma \in \{0.001, 0.003, 0.01\}$. The final settings appear in Table 2. Search records remain in `tuning.csv`.

Scikit-learn trees use the same criterion and stopping settings. SVM baselines use `SGDClassifier` with hinge loss, L2 regularization, `alpha` $= \lambda$, matched epochs, `tol=None` and seed 42. A `Pipeline` includes scaling and, where needed, RFF. Its default optimal learning schedule and lack of the same explicit projection mean that this is a matched-objective comparison, not an identical optimizer [5].

Two additional regression baselines predict the training mean (324,292.79 AZN) or median (210,000 AZN) for every held-out listing. They require no held-out statistics. Final models fit the original training partition; validation rows are not added after selection.

Table 2. Settings selected on validation data.

Model	Settings
Regression tree	MSE; depth 18; minimum leaf 20
Classification tree	Gini; depth 10; minimum leaf 5
Linear SVM	$\lambda = 0.01$; 40 epochs
RFF SVM	$\lambda = 0.0001$; 80 epochs; $\gamma = 0.003$; $D = 512$

B. Metrics and cross-validation

Regression metrics are RMSE and MAE in AZN, price-scale R^2, and RMSE of natural-log prices. Classification metrics are precision, recall, F1 and ROC-AUC, with premium as the positive class. AUC uses tree probabilities or SVM decision scores. The NumPy metric functions include tied-rank AUC handling. A high F1 must not be described as the same percentage accuracy.

Three-fold cross-validation runs within the training partition using the fixed selected configurations. Each fold refits preprocessing and recalculates its price-tier threshold. Fold means and sample standard deviations are reported separately from the single test result. They describe variation within training data; they are neither test confidence intervals nor nested model-selection estimates.

The validation search determines the recorded final settings. However, historical test results were already inspected during development, before the search was extended. Therefore, the test partition is not a fresh blind evaluation. This limitation remains even though the selection code ranks candidates using validation scores only.

C. Correctness and repeatability

The built-in `-check` command covers tiny tree comparisons with scikit-learn, stopping parameters, refitting and class probabilities. SVM checks cover a hand-calculated update, repeatability and objective behaviour. Further checks cover metrics, preprocessing state, split handling and the RFF kernel approximation. These checks passed in the recorded project workflow; they are finite sanity checks, not a proof of correctness for every input.

The reported run uses Python 3.11.9, NumPy 1.26.4, pandas 2.2.2, matplotlib 3.8.4 and scikit-learn 1.4.2 on Windows. The latter four versions are pinned in `requirements.txt`. Transitive versions, including SciPy and threadpoolctl, are recorded in metadata but are not a complete environment lock. One BLAS thread is used. Fit and prediction times exclude shared cleaning and feature preparation; SVM timing includes its scaling and mapping. Timings are a single run and vary with hardware and load.

VI. Results

A. Test metrics and training-fold variation

Table 3 reports all classification models. The NumPy tree reaches F1 0.9068 and AUC 0.9594. Its F1 differs from the scikit-learn tree by less than 0.0002. Small prediction differences are consistent with threshold precision and split tie-breaking, though numerical similarity alone does not establish their exact cause. The linear SVM reaches F1 0.8553, above the RFF model's 0.8478 for the tested configurations. The small gaps between implementations do not establish statistical superiority.

Table 4 shows almost identical regression metrics for the two trees. The NumPy tree improves test MAE by 35.5% over the median baseline and reduces log-RMSE from 0.8360 to 0.4102. Validation also supports useful learning: tree log-RMSE is 0.3814, compared with 0.8035 for the training-median baseline. Nevertheless, price-scale RMSE remains 6.18 million AZN and R^2 is only 0.0105. Log-scale success does not imply accurate predictions for the most expensive listings.

Table 5 separates training-fold variation from test performance. The classification tree has stable F1 across the three folds. RFF F1 varies more than linear SVM F1. Regression CV log-RMSE is 0.4020, close to test log-RMSE, while its CV price-scale R^2 is 0.4273 ± 0.0150. The much lower test R^2 reflects sensitivity to the different target distribution and extreme test values.

The custom regression tree takes 18.081 seconds to fit, compared with 1.773 seconds for scikit-learn. Classification tree times are 9.754 and 1.688 seconds. The NumPy code prioritizes readable algorithm steps, while scikit-learn benefits from optimized tree routines. SVM fit times are closer in this run. They must not be interpreted as general speed guarantees.

B. Validation behaviour

Figure 1 shows the tree trade-off. At depth 18, minimum leaf 5 obtains training log-RMSE 0.2693 but validation log-RMSE 0.4023. Increasing the leaf minimum to 20 raises training error to 0.3448 and lowers validation error to 0.3814. This is evidence that the smaller leaves fit training detail that does not transfer as well.

For classification, Gini at depth 10 and leaf 5 gives validation F1 0.9095. The best entropy candidate reaches 0.9036 at depth 10 and leaf 20. The selected linear and RFF SVMs reach validation F1 0.8577 and 0.8547. RFF does not improve this task over the selected linear model. The search changes feature dimension, bandwidth and epochs together, so it is not a controlled estimate of each factor's separate effect.

C. SVM objective, margin and support mask

Figure 2 shows the recorded objectives. Both begin at 1.0. The linear model ends at 0.3704 and the RFF model at 0.4096. RFF initially overshoots and later fluctuates around a lower level; the curve is not monotonic and does not prove exact convergence to the optimum.

The linear model has $\|w\| = 2.0043$, margin width 0.9979 and 17,592 margin-active training points (38.99%). RFF has $\|w\| = 28.1304$, width 0.0711 and 20,326 such points (45.05%). These widths belong to different feature spaces and should not be compared as physical distances between properties.

A separate 800-row, two-feature illustration studies standardized log area and rooms. At $\lambda = 0.001$, increasing Pegasos training from 10 to 80 epochs lowers objective from 0.5466 to 0.4919. With 80 epochs, raising λ to 0.1 widens the margin from 0.7572 to 1.7218. The alternative inverse-square-root schedule at $\lambda = 0.001$ gives objective 0.5426 and

Table 3. Classification on the fixed test partition. Times are seconds from one recorded run.

Model	Precision	Recall	F1	ROC-AUC	Fit (s)	Predict (s)
NumPy classification tree	0.8966	0.9171	0.9068	0.9594	9.754	0.052
scikit-learn classification tree	0.8970	0.9171	0.9069	0.9593	1.688	0.008
NumPy linear SVM	0.8502	0.8604	0.8553	0.9320	2.842	0.026
scikit-learn linear SVM	0.8477	0.8612	0.8544	0.9322	2.379	0.024
NumPy RFF SVM	0.8237	0.8733	0.8478	0.9275	10.751	0.290
scikit-learn RFF SVM	0.8318	0.8548	0.8432	0.9276	10.416	0.294

Table 4. Regression on the fixed test partition. AZN metrics retain the full target distribution.

Model	RMSE (AZN)	MAE (AZN)	R^2	log-RMSE	Fit (s)	Predict (s)
NumPy regression tree	6,184,534	185,430	0.0105	0.4102	18.081	0.032
scikit-learn regression tree	6,184,534	185,423	0.0105	0.4101	1.773	0.006
Training mean	6,218,064	318,741	−0.0002	0.9249	—	—
Training median	6,220,915	287,621	−0.0012	0.8360	—	—

Table 5. Three-fold training CV: mean ± sample standard deviation. Regression uses log-RMSE; classification uses F1.

Model	Score
NumPy linear SVM	0.8563 ± 0.0026
NumPy RFF SVM	0.8390 ± 0.0115
NumPy classification tree	0.9042 ± 0.0006
NumPy regression tree	0.4020 ± 0.0064
scikit-learn linear SVM	0.8570 ± 0.0028
scikit-learn RFF SVM	0.8435 ± 0.0028
scikit-learn classification tree	0.9042 ± 0.0004
scikit-learn regression tree	0.4018 ± 0.0064

Table 6. Custom-model test confusion counts. Positive class: premium.

Model	TN	FP	FN	TP
Tree	4,346	509	399	4,415
Linear SVM	4,125	730	672	4,142
RFF SVM	3,955	900	610	4,204

width 1.4968. These are sensitivity diagnostics, not scores from the final 179-feature models.

VII. Error Analysis and Limitations

A. Classification errors

The NumPy tree produces 4,346 true negatives, 509 false positives, 399 false negatives and 4,415 true positives. Table 6 lists the confusion counts for the three custom classifiers. The RFF model detects more premium listings than the linear SVM, but also marks more standard listings as premium, reducing precision and F1.

For 531 test listings whose location appears fewer than 100 times in training, F1 falls to 0.7740 for the tree, 0.7101 for linear SVM and 0.4322 for RFF. Limited location coverage is therefore an important weakness, particularly for RFF. Group sizes and class balance differ, so small-group F1 should be interpreted with their counts rather than as a precise ranking of locations.

Table 7. NumPy regression tree errors by price group.

Test group	Rows	MAE (AZN)	log-RMSE
All	9,669	185,430	0.4102
Above 520,000	994	1,465,850	0.7146
At or below 520,000	8,675	38,717	0.3592

B. Extreme targets and regression errors

Table 7 uses the training 90th-percentile price boundary of 520,000 AZN. The 994 test rows above that boundary account for 99.97% of total squared error, computed as group size times squared group RMSE divided by the corresponding total. Their MAE is 1,465,850 AZN. The other 8,675 rows have MAE 38,717 AZN, but their R^2 is still only 0.0422. Therefore, the regression limitation is not fully resolved by describing the expensive tail.

Source inspection finds a 200 m^2 apartment listed at 600,000,000 AZN and a 50 m^2, seven-room apartment at 60,000,000 AZN. These values already occur in the original CSV. The restated price fields agree, which confirms internal consistency but not factual accuracy. A land record contains 200,000 sot, correctly converted by the code to 20,000,000 m^2; the source area itself remains questionable.

The pipeline does not guess corrected prices or remove test examples because they cause large errors. A future study should verify anomalous records using independent evidence, define a cleaning policy before evaluation, and use a fresh evaluation sample. Retaining the records makes the current main metrics transparent, but does not make the source values reliable.

C. Feature interpretation

Area has the largest tree impurity importance: 0.5653 for regression and 0.5796 for classification. Latitude and longitude contribute 0.1245 and 0.1152 in regression. Classification also uses longitude (0.1421), land area (0.0779) and latitude (0.0719). The linear SVM's largest positive standardized weights include rooms (1.1788), area (0.8149) and the new-building category (0.4044). The land category has weight −0.4798.

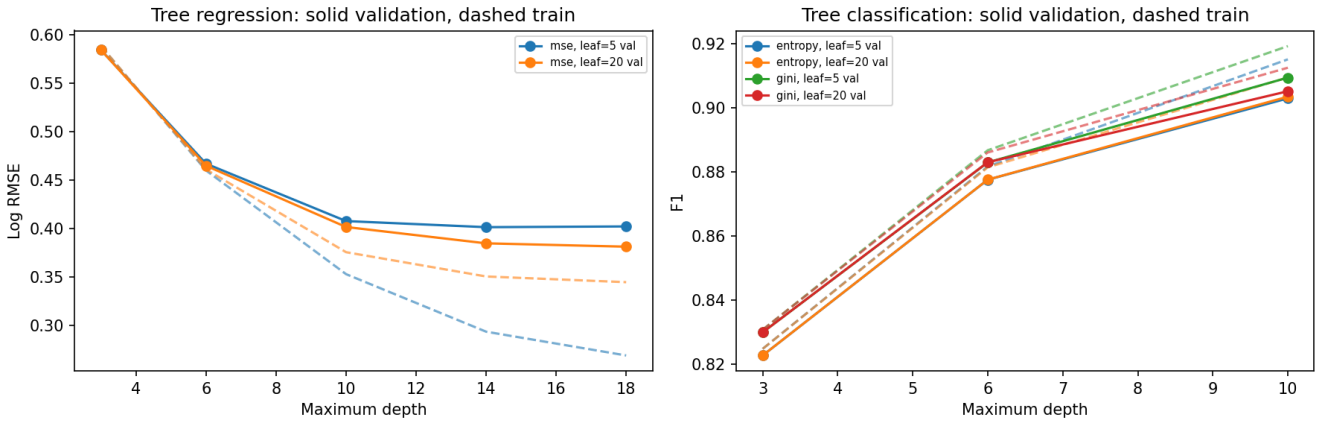


Figure 1. Tree training and validation curves. Dashed curves show training scores; solid curves show validation scores. Lower log-RMSE is better, while higher F1 is better.

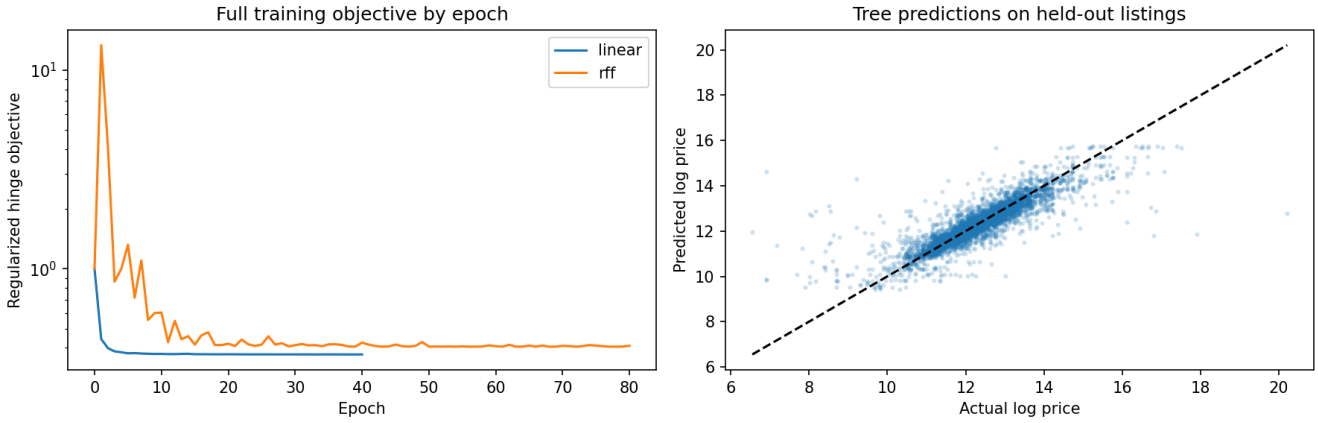


Figure 2. Left: full-training SVM objectives by epoch, with a logarithmic vertical axis. Right: actual and predicted natural-log prices for the regression tree on test rows. Extreme high prices lie far below the equality line.

These associations match the models' use of size, location and property type. They are conditional on encoding and scaling and are not causal explanations. Correlated features can share importance, and impurity importance can favour features with more candidate split points. The full ranked plots are stored in `feature_effects.png`.

D. *Scope of the conclusions*

The label is relative to a sample median, not an official definition of luxury housing. Advertised prices differ from final transaction values. A random split of one historical scrape does not establish performance on future market conditions or new regions. Listing-level deduplication does not guarantee property-level independence.

The mean-log leaf prediction targets a different loss from squared AZN error. Exponentiating it without correction can underpredict arithmetic mean price. Predictor clipping can also suppress genuine extreme characteristics. Neither clipping bounds nor preprocessing choices were tested in a separate ablation, so their individual effects are unknown. Historical test inspection, the limited search and only three CV folds further limit generalization claims.

VIII. Reproducibility and Team Responsibilities

From the repository root, install `requirements.txt`, place the source CSV at `data/house_sale.csv`, and run:

```
python -m src.run_all -check
python -m src.run_all -data data/house_sale.csv
```

The second command regenerates figures and CSV results under `report/figures/`. The data download instructions remain in `data/README.md`. Raw listings are not included in the submission archive. Metadata records the dataset SHA-256, seed, feature count, software versions and selected configurations. Tables here use the stored full-run results; timings may change on a repeat run. Compile this source from the `report` directory with `tectonic report.tex` after generating figures.

The assigned responsibilities are data preparation, leakage checks and pipeline integration for Yagub Khalilli; tree implementation review, baseline checks and error analysis for Ruslan Sadigov; and SVM/RFF review, metrics and cross-validation for Elmir Esgerov. All members share code review, report review and defence preparation. This allocation describes ownership, not a verified record of individual completed contributions. The final `contribution_report.md` and repository history should record the work each member actually completes.

IX. Conclusion

The NumPy models give results close to the matched scikit-learn baselines. The classification tree performs best among the tested custom classifiers, with test F1 0.9068. The regression tree improves substantially over constant predictors on log-RMSE and MAE, but its low price-scale R^2 prevents a claim of reliable valuation across the full dataset. The most useful next steps are evidence-based source validation and evaluation on a new time period. Clear data handling and honest reporting are as important as the model score.

Tools and Acknowledgements

The project uses the supplied course starter pack, the Kaggle dataset and the cited algorithm references. ChatGPT provided technical assistance during development, including method explanations and code review.

References

[1] S. Memmedli, "Bina.az Sale Project," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project. Accessed: Oct. 8, 2026.

[2] scikit-learn developers, "Decision trees," scikit-learn 1.4.2 documentation. [Online]. Available: https://scikit-learn.org/1.4/modules/tree.html. Accessed: Oct. 8, 2026.

[3] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[4] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, pp. 1177–1184, 2007.

[5] scikit-learn developers, "SGDClassifier," scikit-learn 1.4.2 documentation. [Online]. Available: https://scikit-learn.org/1.4/modules/generated/sklearn.linear_model.SGDClassifier.html. Accessed: Oct. 8, 2026.